PROJECT RECON

RAPID EVIDENCE COLLECTION & OSINT NETWORK

"One search. Every tool. No mercy."

VERSION 3.3.0 | 68+ TOOLS | 23 CATEGORIES | MACOS

Complete User Guide & Owner Manual

Installation · Core Commands · SPOTLIGHT Module · All 68 Tools ·

Chain of Custody · Evidence Bags · OpSec · Workflows · Templates

Classification: Sensitive — Authorized Investigator Use Only
Platform: macOS 12+ (Monterey or newer) | **Python:** 3.10+
Generated: March 2026 | **License:** CC BY-SA 4.0

TABLE OF CONTENTS

1 Introduction & Overview 3

What is Project RECON? 3

Core Capabilities at a Glance 3

Legal & Ethical Notice 3

2 System Requirements & Installation 4

Requirements 4

One-Command Install (Portable Drive) 4

Install from GitHub 4

Manual / Developer Install 4

Verifying Installation 4

Uninstalling 4

3 Architecture & Directory Structure 5

File Layout 5

User Data Directory (~/.recon/) 5

Module Overview 5

4 Quick Start 6

First Run 6

Basic Scans 6

Understanding the Output 6

5 Subcommands Reference 7

recon <target> 7

inventory, learn, suggest, categories 7

workflows, templates, update, shell 7

plugins 7

6 Complete Flag Reference 8

Core Flags 8

Scan Control Flags 8

Input Flags 8

Output Flags 9

Investigation Flags 9
Chain of Custody Flags 9
OpSec Flags 9
Automation Flags 9
7 Scan Templates 10
All 14 Built-In Templates 10
8 Investigation Workflows 11
All 11 Workflows 11
9 SPOTLIGHT — Relationship Investigation Module 12
Overview & Commands 12
The 13 Investigation Phases 12
SPOTLIGHT Flags Reference 13
SPOTLIGHT Examples 13
10 Tool Catalog — All 68 Tools 14
Username Enumeration (7 tools) 14
Email Intelligence (3 tools) 15
Phone Intelligence (2 tools) 15
Network Forensics (7 tools) 15
Metadata & File Analysis (7 tools) 16
And all remaining categories… 17
11 Session Tracking 22
12 Chain of Custody & Evidence Bags 23
How It Works 23
MANIFEST.json & CHAIN_OF_CUSTODY.json 23
Verifying Integrity 23
13 OpSec & Anonymity 24
14 Report Export 25
15 Auto-Update System 25
16 Plugin System & Marketplace 26
17 Troubleshooting 27

18 Forensics Glossary 28

CHAPTER 1

INTRODUCTION & OVERVIEW

What Project RECON is, what it does, and who it is for.

What Is Project RECON?

Project RECON (**Rapid Evidence Collection & OSINT Network**) is a unified, command-line investigation workbench for macOS. It replaces the need to manually install, configure, and run dozens of separate open-source intelligence (OSINT) and digital forensics tools by orchestrating all of them through a single interface.

Built on top of the **WhatsMyName** project (a community-maintained database of 4,387+ sites that can be searched for usernames), RECON extends it into a full investigation platform with sessions, chain-of-custody evidence packaging, exposure scoring, relationship investigation, face recognition, and operational security controls.

> ℹ️ **Built for Investigators**
>
> Project RECON is designed for use by licensed private investigators, law enforcement, corporate security teams, and individuals conducting legal OSINT investigations. All outputs are structured for potential use as documented evidence.

Core Capabilities at a Glance

🔍 OSINT Orchestration

- 68+ tools across 23 categories
- Auto-detects input type (username / email / phone)
- Runs tools in parallel — live progress dashboard
- Deduplicates findings across all tools
- 4,387+ site database (WMN + Sherlock + Maigret)

💑 SPOTLIGHT Module

- 35+ relationship investigation tools
- 13 investigation phases
- Instagram deep OSINT (4 tools)
- Dating & adult platform crawling
- Behavioral metadata & geolocation

⚖️ Forensic Standards

- SHA-256 hash verification

🛡️ Operational Security

- VPN detection (refuse to run if unprotected)

- Chain-of-custody logging
- Evidence bag packaging
- Timestamped action logs
- MANIFEST.json + CHAIN_OF_CUSTODY.json

- Tor routing — built-in SOCKS5 support
- Rate limiting + random jitter delays
- OpSec warnings before active scans
- Proxy support for all tools

Legal & Ethical Notice

⚠️ **Important — Read Before Use**

Project RECON performs open-source intelligence collection from publicly available sources. You are responsible for ensuring your use complies with all applicable laws in your jurisdiction, including the Computer Fraud and Abuse Act (CFAA), GDPR, CCPA, and relevant state/local statutes.

Never use this tool to: harass individuals, stalk persons without legal authority, access private systems without permission, or collect data on minors. Always obtain required legal authorization before conducting investigations on third parties.

CHAPTER 2

SYSTEM REQUIREMENTS & INSTALLATION

Everything you need to get Project RECON running on your machine.

Requirements

REQUIREMENT	MINIMUM	RECOMMENDED
Operating System	macOS 12 Monterey	macOS 14 Sonoma or newer
Python	3.10	3.12+
Homebrew	Not required (but recommended)	Installed
Disk Space	500 MB	2 GB (full tool suite)
RAM	4 GB	8 GB+
Internet	Required for scanning	Stable broadband
Docker	Optional	Required for CompreFace face recognition

One-Command Install (Portable Drive)

If RECON ships on a portable drive, plug it in and run:

```
bash /Volumes/YOUR_DRIVE/RECON_WorkBench/install.sh
```

The installer will automatically:

- Copy Project RECON to `~/RECON_WorkBench`
- Create an isolated Python virtual environment at `~/RECON_WorkBench/.venv`
- Install all 40+ Python OSINT tools via pip
- Install Homebrew tools (nmap, tshark, tcpdump, exiftool, etc.) if Homebrew is present
- Add the `recon` and `spotlight` commands to your shell PATH
- Set up a weekly auto-update job (Mondays at 3:00 AM)

Install from GitHub

```
git clone https://github.com/newbiehackler/project-recon.git
cd project-recon
bash install.sh
```

Manual / Developer Install

```
git clone https://github.com/newbiehackler/project-recon.git
cd project-recon
python3 -m venv .venv && source .venv/bin/activate
pip install -e .

# Core OSINT tools
pip install sherlock-project maigret social-analyzer socialscan holehe h8mail nexfil

# SPOTLIGHT tools
pip install ghunt instaloader xeuledoc snscrape pyktok redditsfinder telepathy
pip install osrframework crosslinked linkedin2username daProfiler photon-crawler toutatis
```

Verifying Installation

```
recon --version        # Should print version 3.3.0
recon --list-tools     # Shows all tools and their install status
recon inventory        # Full system audit
```

Uninstalling

```
bash ~/RECON_WorkBench/uninstall.sh
```

This removes RECON, the `recon` / `spotlight` launchers, auto-update jobs, and shell PATH entries. Your Homebrew and pip tools remain untouched — only RECON-specific components are removed.

CHAPTER 3

ARCHITECTURE & DIRECTORY STRUCTURE

Understanding how Project RECON is organized internally.

File Layout

```
~/RECON_WorkBench/
├── install.sh              # One-command portable installer
├── update.sh               # Auto-update script (called by launchd weekly)
├── uninstall.sh            # Clean removal script
├── whatsmyname/            # Core Python package
│   ├── recon_cli.py        # Main CLI entry point — all flags parsed here
│   ├── orchestrator.py     # Tool execution engine (parallel runner)
│   ├── tool_catalog.py     # 68 tool definitions, 10+ examples each
│   ├── workflows.py        # 11 investigation workflow definitions
│   ├── templates.py        # 14 scan template profiles
│   ├── spotlight.py        # SPOTLIGHT relationship investigation CLI
│   ├── sessions.py         # Session persistence (<name>.json in ~/.recon/sessions/)
│   ├── evidence.py         # EvidenceBag: SHA-256 hashing, chain of custody
│   ├── enrichment.py       # DNS/IP enrichment of findings
│   ├── reporter.py         # JSON / CSV / HTML report generation
│   ├── shell.py            # Interactive REPL shell
│   ├── plugin_loader.py    # Plugin discovery and loading
│   └── data/
│       ├── wmn-data.json           # 731 core WMN sites
│       └── extended_sites.json    # 1,076 curated additions
├── tools/                  # Bundled git-cloned tools
│   ├── blackbird/          # Username search (600+ sites)
│   ├── cupidcr4wl/         # Dating/adult platform OSINT
│   ├── compreface/         # Face recognition via Docker
│   ├── untappd-scraper/    # Social behavior & geolocation intel
│   ├── 4n6notebooks/       # iOS forensic Jupyter notebooks
│   ├── keychain-decrypter/ # iOS keychain decryption
│   ├── osint-tools-cli/    # 1,000+ OSINT tool browser (Rust TUI)
│   ├── osintgram/          # Instagram deep OSINT
│   ├── instaloctrack/      # Instagram geolocation mapping
│   ├── pwnedornot/         # HIBP breach checker + password recovery
│   ├── metagoofil/         # Public document metadata extractor
│   └── spiderfoot/         # 200+ module OSINT automation
└── wmn-data.json           # WhatsMyName site database (4,387+ sites)
```

User Data Directory (~/.recon/)

```
~/.recon/
├── plugins/            # Drop .py plugin files here — auto-loaded on start
├── sessions/           # Investigation sessions as JSON files
├── spotlight_plugins/  # SPOTLIGHT module plugins
├── evidence/           # Evidence bags from --evidence-bag runs
├── exports/            # Exported reports (JSON, HTML, CSV, PDF)
└── logs/               # Auto-update logs
```

Module Overview

MODULE	PURPOSE
`recon_cli.py`	Argument parsing, flag routing, main entry point for `recon` command
`orchestrator.py`	Launches tools as subprocesses, manages parallel execution, semaphore limiting, retries, proxy injection, jitter
`tool_catalog.py`	Defines all 68 tools: name, command, category, description, install_hint, input_types, passive flag, examples, works_well_with
`evidence.py`	EvidenceBag class: store_artifact(), store_raw_output(), verify_integrity(), finalize() → writes MANIFEST.json + CHAIN_OF_CUSTODY.json
`sessions.py`	Session creation, resume, history, finding aggregation — stored as JSON in ~/.recon/sessions/
`enrichment.py`	DNS/IP/reverse-DNS enrichment of scan findings
`templates.py`	14 ScanTemplate objects: defines tool lists, timeout multipliers, passive/deep/quick flags
`workflows.py`	11 investigation workflow definitions with step-by-step tool chaining
`spotlight.py`	SPOTLIGHT CLI: 35+ relationship tools, 3 scan modes, 13 phases
`shell.py`	Interactive REPL: history, auto-complete, session management
`plugin_loader.py`	Discovers, validates, and loads plugins from ~/.recon/plugins/

CHAPTER 4

QUICK START

Get your first scan running in under 60 seconds.

First Run

After installation, update the site database to get the latest detection data from Sherlock and Maigret:

```
recon update
```

This fetches ~3,500 additional site definitions and caches them locally. After deduplication you will have **4,387+ unique sites** to search against.

Basic Scans

Username Search

```
recon johndoe
```

RECON auto-detects that `johndoe` is a username and runs all applicable tools in parallel. It also generates email variants (`johndoe@gmail.com`, `@yahoo.com`, etc.) and runs email intelligence tools against those automatically.

Email Search

```
recon johndoe@gmail.com
```

Runs: holehe (service registrations), h8mail (breach databases), socialscan (platform availability), theHarvester (domain intelligence).

Phone Number Search

```
recon +15551234567
```

Runs: phoneinfoga (carrier/location/search links), ignorant (WhatsApp/Telegram/Signal/Snapchat registration check).

Common Useful Commands

```
# Deep scan — all tools, 2× timeouts
recon johndoe --deep

# Export as HTML report and open in browser
recon johndoe -o html --open

# Quick scan (top 3 fastest tools, ~60 seconds)
recon johndoe --quick

# Use a predefined template
recon johndoe --template full-osint

# Track in a named session with examiner tag
recon johndoe --session case001 --case-id CASE-2024-001 --examiner "Agent Smith"

# Dry run — preview what WOULD execute without running anything
recon johndoe --dry-run

# Suggest next tools after scan
recon johndoe --discover

# Calculate exposure risk score (1-100)
recon johndoe --score
```

Understanding the Output

When a scan completes, RECON displays:

- **Live progress bar** — shows each tool running, elapsed time, findings per tool
- **Summary table** — total unique accounts found, tools completed/failed, duration
- **Findings grouped by category** — Social, Dating, Finance, Gaming, etc.
- **Exposure score** (if `--score`) — 1–100 risk rating with category breakdown
- **Next-step suggestions** (if `--discover`) — ranked tool recommendations

📄 JSON Report Structure


```
{
  "target": "johndoe",
  "input_type": "username",
  "timestamp": "2024-01-15T14:30:00+00:00",
  "summary": {
    "total_unique_accounts": 47,
    "tools_completed": 8,
    "tools_failed": 0,
    "elapsed_seconds": 68.3
  },
  "tools": [
    {"name": "wmn",      "status": "completed", "findings": 23, "elapsed": 45.2},
    {"name": "sherlock", "status": "completed", "findings": 18, "elapsed": 30.1}
  ],
  "findings": [
    {"source": "wmn", "site": "Instagram", "url": "https://instagram.com/johndoe", "category":
"social"},
    {"source": "sherlock", "site": "GitHub", "url": "https://github.com/johndoe", "category":
"coding"}
  ]
}
```

CHAPTER 5

SUBCOMMANDS REFERENCE

Every subcommand available in the `recon` CLI.

SUBCOMMAND	DESCRIPTION	EXAMPLE
`recon <target>`	Scan a username, email, or phone number	`recon johndoe`
`recon inventory`	Full system audit — shows all 68 tools with install status, version where detectable, and category	`recon inventory`
`recon learn <tool>`	Show beginner-friendly guide for a specific tool: description, 10+ examples, install hint, and next-step suggestions	`recon learn sherlock`
`recon suggest <tool>`	Get ranked recommendations for what to run next after using a given tool, with reasoning	`recon suggest holehe`
`recon categories`	Browse all 23 tool categories with tool lists	`recon categories`
`recon workflows`	List all 11 investigation workflows with step breakdowns	`recon workflows`
`recon templates`	List all 14 scan templates with tool lists and flags	`recon templates`
`recon update`	Update all tools and site database	`recon update --pip`
`recon shell`	Launch the interactive REPL investigation shell with history and auto-complete	`recon shell`
`recon explain <term>`	Explain a forensics concept or term (chain-of-custody, OSINT, passive, active, SHA-256, etc.)	`recon explain "chain of custody"`
`recon plugins`	List, search, install, create, validate, and submit plugins	`recon plugins search`

recon update — Subcommand Options

```
recon update              # Update everything (pip + brew + bundled tools + site data)
recon update --pip        # Only update Python pip tools
recon update --brew       # Only update Homebrew tools
recon update --bundled    # Only update git-cloned bundled tools (git pull)
```

recon plugins — All Plugin Subcommands

```
recon plugins                      # List installed plugins
recon plugins search               # Browse all marketplace plugins
recon plugins search "username"    # Search marketplace for keyword
recon plugins install social-deep  # Install a free plugin
recon plugins install premium --key RECON-PREM-abc123  # Install paid plugin
recon plugins uninstall my-plugin  # Remove a plugin
recon plugins create my-scanner    # Scaffold a new plugin with boilerplate
recon plugins validate my.py       # Validate plugin before submission
recon plugins submit my.py         # Submit to marketplace via GitHub Issue
```

recon learn — What It Shows

Running `recon learn <toolname>` displays a structured guide for any of the 68 tools, including:

- Full description of the tool
- Install instructions (`brew install`, `pip install`, or git clone)
- 10+ practical command examples with descriptions
- What input types the tool accepts (username / email / phone / file / IP)
- Whether the tool is passive (no direct target contact) or active
- Recommended "next tools" to chain with

```
recon learn nmap          # Network scanner guide
recon learn exiftool      # Metadata extraction guide
recon learn toutatis      # Instagram OSINT guide
recon learn john          # Password cracking guide
```

CHAPTER 6

COMPLETE FLAG REFERENCE

Every flag available in `recon <target> [flags]`, organized by function.

Core Flags

FLAG	DESCRIPTION	EXAMPLE
`--type username|email|phone`	Override input auto-detection. Useful when input is ambiguous.	`--type email`
`--tools wmn,sherlock`	Comma-separated list of specific tools to run. All others are skipped.	`--tools holehe,h8mail`
`--timeout 120`	Per-tool timeout in seconds. Default varies by tool.	`--timeout 300`
`-o json|csv|html`	Export report format. File named <target>.<ext> by default.	`-o html`
`--output-file path`	Custom output file path (overrides default naming).	`--output-file /tmp/case001.json`
`--no-banner`	Skip the ASCII banner (useful for scripting/automation).	`--no-banner`
`--list-tools`	Show all 68 tools with install status and skip the scan.	`--list-tools`
`-V, --version`	Print version and exit.	`-V`

Scan Control Flags

FLAG	DESCRIPTION
`--deep`	Run all applicable tools with 2× timeouts. Most thorough, slowest mode.
`--quick`	Run only the 3 fastest tools. Under 60 seconds for a result.
`--passive`	Non-intrusive tools only — tools flagged as passive in the catalog. Does not directly contact the target.
`--dry-run`	Preview which tools would run and with what arguments, without executing anything.
`--template <name>`	Use a predefined scan template (see Chapter 7 for full list).
`--parallel N`	Maximum number of tools to run concurrently. Default: unlimited. Useful for rate-limited connections.
`--retries N`	Number of times to retry a failed tool. Default: 0.
`--proxy URL`	Route all tool requests through an HTTP/SOCKS proxy.
`--tor`	Route through Tor (requires Tor running on port 9050). Builds `socks5://127.0.0.1:9050` proxy URL automatically.

Input Flags

FLAG	DESCRIPTION
`-f targets.txt`	Read multiple targets from a file (one per line). Runs a full scan for each.
`--stdin`	Read targets from stdin. Useful for piping: `cat targets.txt | recon --stdin`
`--no-email-expand`	Skip automatic generation of email variants for username searches.

Output Flags

FLAG	DESCRIPTION
`--raw`	Print full raw stdout and stderr from each tool after the scan completes.
`--save-raw ./dir`	Save per-tool stdout and stderr to a directory as separate text files.
`--no-color`	Disable all ANSI color codes. Use for logging, CI/CD pipelines, or accessibility.
`--open`	Automatically open the HTML report in your default browser after export.
`--pdf`	Export report as PDF (requires weasyprint or wkhtmltopdf).
`--markdown`	Export the scan report as a Markdown (`.md`) file.

Investigation Flags

FLAG	DESCRIPTION
`--session <name>`	Create or resume a named investigation session. All findings are appended.
`--case-id ID`	Tag the scan with a formal case identifier (e.g., CASE-2024-001). Stored in session and reports.
`--notes "text"`	Attach investigator notes to the session.
`--examiner "Name"`	Tag the examiner's name. Appears in evidence bag and chain of custody.
`--discover`	After the scan, show ranked suggestions for which tools to run next and why.
`--enrich`	Enrich findings with DNS lookups, IP geolocation, and reverse DNS data.
`--score`	Calculate a 1–100 exposure risk score with breakdown by category.
`--correlate`	Cross-reference findings across multiple tools — identify URLs appearing in multiple sources.
`--diff prev.json`	Compare current scan against a previous JSON export. Shows new and removed findings.
`--filter "cat=social"`	Filter findings by category, tool, confidence, or site. E.g. `--filter category=dating`

Chain of Custody Flags

FLAG	DESCRIPTION
`--evidence-bag ./dir`	Create a forensic evidence bag directory. All tool outputs, reports, and raw data are stored with SHA-256 hashes.
`--chain-of-custody`	Generate a timestamped CHAIN_OF_CUSTODY.json alongside every evidence artifact.
`--hash-verify`	After writing outputs, verify all file hashes match recorded values. Alerts on any tampering.

OpSec Flags

FLAG	DESCRIPTION
`--opsec-check`	Before running, warn about any active (non-passive) tools that will directly contact the target. Prompts for confirmation.
`--vpn-check`	Detect VPN presence (checks for utun/tun/wg network interfaces). Exit with error if no VPN detected.
`--rate-limit N`	Maximum requests per second across all tools. Prevents triggering rate-limit bans.
`--jitter`	Add random delays between tool launches (0.5–3 seconds). Mimics human behavior, reduces detection risk.

Automation Flags

FLAG	DESCRIPTION
`--webhook URL`	POST a JSON summary to a webhook URL when the scan completes. For integration with Slack, Discord, Zapier, etc.
`--notify`	Send a macOS desktop notification when the scan finishes.
`--on-finding "cmd {}"`	Execute a shell command for each finding. `{}` is replaced with the finding URL. E.g. `--on-finding "echo Found: {}"`
`--explain "term"`	Print a plain-English explanation of a forensics term. Can also be used as: `recon explain "chain of custody"`
`--cheatsheet <tool>`	Quick reference card for a tool — condensed examples, no full guide.

CHAPTER 7

SCAN TEMPLATES

Predefined scan profiles for common investigation types. Use with `recon <target> --template <name>`

TEMPLATE NAME	DESCRIPTION	MODE	TOOLS
`full-osint`	Maximum coverage — all applicable tools, 2× timeouts, email expansion	Deep	All
`passive-only`	Non-intrusive only — tools that don't directly contact the target	Passive	wmn, sherlock, maigret, social-analyzer, socialscan, nexfil, holehe, h8mail, ignorant, theHarvester
`quick-check`	Fast 60-second scan — top 3 fastest tools only	Quick	wmn, sherlock, socialscan
`breach-check`	Focus on data breaches and email exposure	Normal	holehe, h8mail, socialscan
`username-deep`	Deep username enumeration across all username tools	Deep	wmn, sherlock, maigret, social-analyzer, socialscan, nexfil
`phone-intel`	Phone number investigation — carrier + platform registrations	Normal	phoneinfoga, ignorant
`email-full`	Complete email investigation — registrations + breaches + domain	Normal	holehe, h8mail, socialscan, theHarvester
`relationship`	SPOTLIGHT FULL — complete relationship investigation (35+ tools, 13 phases)	Deep	35+ tools (see Chapter 9)
`dating-sweep`	SPOTLIGHT Dating — focused sweep of dating/hookup/adult platforms	Quick	cupidcr4wl, blackbird, maigret, toutatis, holehe, socialscan, onlyfinder, ignorant
`spotlight-quick`	SPOTLIGHT Quick — 7 highest-impact tools, under 2 minutes	Quick	sherlock, blackbird, cupidcr4wl, holehe, socialscan, ignorant, toutatis
`spotlight-instagram`	SPOTLIGHT Instagram — complete Instagram account investigation	Deep	toutatis, osintgram, instaloader, instaloctrack, sherlock, exiftool
`spotlight-email`	SPOTLIGHT Email — full email identity chain (registrations + breaches + Google + profiling)	Deep	holehe, h8mail, ghunt, infoga, pwnedornot, socialscan, daProfiler, spiderfoot
`spotlight-phone`	SPOTLIGHT Phone — carrier + messaging app registrations + Telegram	Normal	phoneinfoga, ignorant, telepathy, telerecon

Using Templates

```
# Full investigation — no shortcuts
recon johndoe --template full-osint

# Dating platforms only — fast
recon johndoe --template dating-sweep

# Complete relationship investigation
recon johndoe --template relationship --session case001 --examiner "Det. Smith"

# Instagram deep dive with evidence bag
recon johndoe --template spotlight-instagram \
  --evidence-bag ./evidence/ig_case/ --chain-of-custody

# Email breach check
recon target@gmail.com --template breach-check -o json
```

CHAPTER 8

INVESTIGATION WORKFLOWS

Step-by-step tool-chaining guides for common investigation types. View with `recon workflows`

> ℹ️ **What Are Workflows?**
>
> Workflows are ordered sequences of tools, each with a specific purpose. Unlike templates (which run in parallel), workflows are intended to be followed step-by-step — the output of one tool informs the input of the next.

Workflow 1 — Full Username OSINT

1. **wmn** — Enumerate username across 4,300+ sites
2. **sherlock** — Cross-validate with Sherlock's detection engine
3. **maigret** — Deep search with profile data extraction
4. **holehe** — Check email variants for service registrations
5. **h8mail** — Search email variants in breach databases
6. **exiftool** — Extract metadata from any discovered profile images

Workflow 2 — Email Intelligence

1. **holehe** — Check which services the email is registered on
2. **h8mail** — Search for the email in data breaches
3. **socialscan** — Verify email availability on major platforms
4. **theHarvester** — Find related emails and subdomains from the domain
5. **whois** — Look up the email's domain for registrant info
6. **dig** — Check DNS records (MX, SPF, DKIM) for the domain

Workflow 3 — Phone Number Investigation

1. **phoneinfoga** — Get carrier, line type, and location data

2. **ignorant** — Check which platforms the number is registered on (WhatsApp, Telegram, Signal, Snapchat)
3. **holehe** — If you find an associated email, check its service registrations
4. **sherlock** — If you find a username, enumerate profiles across platforms

Workflow 4 — Network Reconnaissance

1. **ping** — Verify target is alive
2. **traceroute** — Map the network path
3. **whois** — Look up IP/domain ownership
4. **dig** — Enumerate all DNS records
5. **nmap** — Scan for open ports and services
6. **tshark** — Capture and analyze traffic
7. **curl** — Probe discovered web services

Workflow 5 — Domain Investigation

1. **whois** — Look up domain registration and ownership
2. **dig** — Check all DNS record types (A, MX, TXT, NS, CNAME)
3. **theHarvester** — Find emails, hosts, and subdomains
4. **nmap** — Scan the domain's IP for open services
5. **curl** — Check HTTP headers and server technology stack

Workflow 6 — File Forensics

1. **file** — Identify the real file type (never trust the extension)
2. **exiftool** — Extract all metadata (GPS, author, dates, camera model)
3. **strings** — Find readable text hidden in binary data
4. **xxd** — Hex dump to inspect raw bytes for anomalies
5. **steghide** — Check for steganographic data hidden in JPEG/BMP/WAV

Workflow 7 — Passive OSINT Only (Maximum OpSec)

All tools in this workflow are flagged `passive=True` in the tool catalog — they do not directly contact or interact with the investigation target.

1. **wmn** — Username enumeration (passive web checks)
2. **sherlock** — Profile detection (passive web checks)
3. **holehe** — Email registration checks (passive login probes)
4. **whois** — Domain/IP registration data (public records only)
5. **dig** — DNS lookups (public records only)
6. **exiftool** — Metadata extraction (local file analysis, no network)

CHAPTER 9

SPOTLIGHT — RELATIONSHIP INVESTIGATION MODULE

Comprehensive relationship, infidelity, and deception investigation with 35+ tools across 13 phases.

Overview

SPOTLIGHT is a dedicated investigation module within Project RECON, accessible via both the `spotlight` command and `recon --template relationship`. It is designed specifically for **relationship investigations** — situations where you need to determine whether someone is maintaining hidden online presences, secret accounts on dating platforms, or concealed social media activity.

Full Mode (Default)

- 35+ tools across 13 phases
- Thorough and comprehensive
- Runs 45–90 minutes typically
- Covers every platform category

Quick Mode (`--quick`)

- 7 highest-impact tools
- Under 2 minutes
- sherlock, blackbird, cupidcr4wl
- holehe, socialscan, ignorant, toutatis

Dating Mode (`--dating`)

- 8 dating/adult-focused tools
- Covers hookup & adult platforms
- cupidcr4wl, blackbird, maigret
- toutatis, holehe, socialscan, onlyfinder

Input Formats

- Username: `spotlight johndoe`
- Email: `spotlight user@gmail.com`
- Phone: `spotlight +15551234567`
- All formats auto-detected

The 13 Investigation Phases

PHASE	TOOLS	PURPOSE
1 Username Enumeration	wmn, sherlock, maigret, social-analyzer, socialscan, nexfil	Broadest coverage — 4,300+ sites across all categories
2 Dating / Relationship	blackbird, cupidcr4wl	600+ sites including all major dating, hookup, and adult platforms
3 Instagram Deep OSINT	toutatis, osintgram, instaloader, instaloctrack	Extract hidden contact info, full account analysis, geolocation mapping
4 Google Account	ghunt, xeuledoc	Gmail GAIA ID, Maps/YouTube activity, Drive document ownership
5 Telegram & Messaging	telepathy, telerecon, ignorant	Telegram group membership, channel archives, phone platform registration check
6 Social Media Scrapers	snscrape, twint, redditsfinder, pyktok	Twitter/X, Reddit, TikTok archives without API keys
7 Email Intelligence	holehe, h8mail, infoga, pwnedornot	Service registrations, breach databases, leaked password recovery
8 People Profiling	daProfiler, spiderfoot, osrframework	Comprehensive identity dossier from social data, photos, usernames
9 Phone Intelligence	phoneinfoga	Carrier info, line type (mobile/landline/VOIP), Truecaller/Numverify lookup
10 Metadata & Behavior	exiftool, metagoofil, untappd-scraper	GPS in photos, document author metadata, bar/venue check-in geolocation
11 Face Recognition	compreface	Match face from profile photo across all discovered images (requires Docker)
12 Adult Platform Search	onlyfinder	OnlyFans, Fansly, and 8+ other adult content platforms
13 OSINT Framework	recon-ng	Modular correlation and reporting of all discovered data

SPOTLIGHT Flags Reference

FLAG	DESCRIPTION
`--quick`	7-tool fast sweep: sherlock, blackbird, cupidcr4wl, holehe, socialscan, ignorant, toutatis
`--dating`	Dating/adult focus: cupidcr4wl, blackbird, maigret, toutatis, holehe, socialscan, onlyfinder, ignorant
`--tools a,b,c`	Override tool list — run only specified tools
`--timeout N`	Per-tool timeout (default: 120s full, 60s quick)
`--session NAME`	Track in a named investigation session
`--case-id ID`	Tag report with formal case identifier
`--examiner NAME`	Tag examiner name for documentation
`--notes TEXT`	Attach notes to the investigation
`--score`	Calculate exposure risk score (1–100)
`--discover`	Suggest complementary tools after scan
`--raw`	Show raw stdout from each tool
`--no-color`	Plain text output
`--open`	Auto-open HTML report in browser
`--proxy URL`	Route through proxy
`--tor`	Route through Tor (port 9050)
`--jitter`	Random delays between tool launches

SPOTLIGHT Examples

```
# Full relationship investigation (35+ tools)
spotlight johndoe

# Quick 7-tool sweep — under 2 minutes
spotlight johndoe --quick

# Dating platforms only
spotlight johndoe --dating

# Investigate by email address
spotlight johndoe@gmail.com

# Investigate by phone number
spotlight +15551234567

# Full investigation with JSON export
spotlight johndoe -o json --output-file case001.json

# HTML report auto-opened in browser
spotlight johndoe -o html --open

# Named session + formal case tracking
spotlight johndoe --session case001 --case-id CASE-2024-001 --examiner "Detective Smith"

# Maximum OpSec — Tor + jitter
spotlight johndoe --tor --jitter

# Exposure score + next-step suggestions
spotlight johndoe --score --discover

# Instagram deep OSINT only
spotlight johndoe --tools toutatis,osintgram,instaloader,instaloctrack

# Full investigation via recon templates
recon johndoe --template relationship
recon johndoe --template spotlight-instagram
recon target@gmail.com --template spotlight-email
recon +15551234567 --template spotlight-phone
recon johndoe --template dating-sweep
```

CHAPTER 10

TOOL CATALOG — ALL 68 TOOLS

Every tool in Project RECON, organized by category with key examples. Run `recon learn <tool>` for 10+ examples per tool.

Username Enumeration (7 Tools)

wmn — WhatsMyName

The core engine. Checks 4,387+ sites using precise match/no-match response analysis. Highest accuracy of any username tool.

PASSIVE USERNAME PIP INSTALL

```
wmn johndoe                              # Basic scan
wmn johndoe --nsfw                       # Include adult sites
wmn johndoe --category dating,social     # Filter by category
wmn johndoe --output html --workers 200  # Fast scan + HTML report
wmn johndoe --tor --workers 10           # Route through Tor
```

sherlock — Sherlock Project

Username search across 400+ social networks. Excellent coverage of mainstream platforms. Different detection engine from WMN — run both for maximum coverage.

PASSIVE USERNAME PIP INSTALL SHERLOCK-PROJECT

```
sherlock johndoe                         # Basic search
sherlock johndoe --output results.txt    # Save to file
sherlock johndoe --proxy socks5://127.0.0.1:9050  # Via Tor
sherlock johndoe --timeout 30 --print-found  # Only show hits
```

maigret — Deep Username Search

Searches ~3,000 sites and extracts profile data (bio, full name, linked accounts). Supports tag filtering — use `--tags dating` for relationship investigations.

PASSIVE USERNAME PIP INSTALL MAIGRET

```
maigret johndoe                          # Full 3,000-site search
maigret johndoe --tags dating,finance    # Dating sites only
maigret johndoe --html report.html       # Interactive HTML report
maigret johndoe -a --no-progressbar      # All sites, quiet mode
```

social-analyzer — 900+ Sites with Confidence Scoring

Profile detection across 900+ sites using multiple analysis methods. Provides confidence scoring (good / maybe / bad) for each result.

PASSIVE USERNAME PIP INSTALL SOCIAL-ANALYZER

```
social-analyzer --username johndoe --mode fast
social-analyzer --username johndoe --filter good --output json
social-analyzer --username johndoe --method find,extract --output json
```

socialscan — Email/Username Availability

Accurate username and email availability checks on major platforms via API queries. Fastest tool for confirming account existence.

PASSIVE USERNAME PIP INSTALL SOCIALSCAN

```
socialscan johndoe                          # Check username
socialscan johndoe@gmail.com                # Check email
socialscan user1 user2 user3 --json out.json # Multiple targets
```

nexfil — NExfil OSINT

Cross-platform username search with HTTP response code inspection. Good coverage of niche and international platforms.

PASSIVE USERNAME PIP INSTALL NEXFIL

```
nexfil -u johndoe -l results.txt  # Save to file
nexfil -u johndoe -H              # Show HTTP response codes
```

blackbird — 600+ Sites (Dating Focus)

Searches 600+ sites with emphasis on dating, adult content, and relationship platforms. Accepts both username and email. Critical for relationship investigations.

PASSIVE USERNAME/EMAIL RELATIONSHIP

```
blackbird --username johndoe --json
blackbird --email johndoe@gmail.com
blackbird --username johndoe --filter dating --json
blackbird --username johndoe --pdf report.pdf
```

Email Intelligence (3 Tools)

holehe — Service Registration Checker

Checks 500+ services to see if an email address is registered. Uses "forgot password" flows — highly accurate with minimal false positives.

PASSIVE EMAIL PIP INSTALL HOLEHE

```
holehe target@gmail.com --only-used --no-color > results.txt
holehe target@gmail.com --rate-limit 1 --proxy http://127.0.0.1:8080
```

h8mail — Breach Database Lookup

Searches for emails in known data breaches. With API keys, retrieves leaked passwords and associated usernames from premium breach sources.

PASSIVE EMAIL PIP INSTALL H8MAIL

```
h8mail -t target@gmail.com -o results.csv
h8mail -t targets.txt -k config.ini --chase  # Chase related emails
h8mail -t target@gmail.com --power-to-the-user  # Free local breach data
```

theHarvester — Email, Host & Subdomain Gathering

Gathers emails, hosts, URLs, and subdomains from public sources (Google, Bing, DuckDuckGo, HackerTarget, etc.).

PASSIVE EMAIL/DOMAIN PIP INSTALL THEHARVESTER

```
theharvester -d example.com -b all -l 100 -f report
theharvester -d johndoe -b duckduckgo,urlscan -l 50
theharvester -d example.com -b all -l 500 --dns-resolve
```

Phone Intelligence (2 Tools)

phoneinfoga — Phone Number OSINT

Advanced phone number intelligence: carrier, country, line type (mobile/VOIP/landline), and Google dorking scanner.

PASSIVE PHONE

```
phoneinfoga scan -n +15551234567 --output json > phone.json
phoneinfoga scan -n +15551234567 --scanner googleSearchFootprint
phoneinfoga scan -n +15551234567 --scanner local  # Offline only
```

ignorant — Phone Platform Registration

Checks whether a phone number is registered on WhatsApp, Telegram, Signal, Snapchat, and other messaging platforms.

PASSIVE PHONE PIP INSTALL IGNORANT

```
ignorant +1 5551234567 --only-used --no-color > ignorant_results.txt
ignorant +44 7911123456 --timeout 20 --proxy http://127.0.0.1:8080
```

Network Forensics (5 Tools)

nmap — Network Scanner

The gold standard network scanner. Discovers hosts, ports, services, OS fingerprints, and vulnerabilities.

ACTIVE NETWORK BREW INSTALL NMAP

```
nmap -sV 192.168.1.1             # Service/version detection
nmap -A -T4 target.com           # Aggressive: OS + scripts + traceroute
nmap -p 1-65535 -T4 192.168.1.1  # Full port scan
nmap --script vuln 192.168.1.1   # Vulnerability scripts
nmap -sn 192.168.1.0/24          # Ping sweep subnet
```

tshark — Command-Line Wireshark

Capture and analyze network packets from the terminal. Full Wireshark dissector library available.

PASSIVE NETWORK BREW INSTALL WIRESHARK

```
tshark -i en0 -w output.pcap -a duration:60
tshark -r capture.pcap -Y 'dns' -T fields -e dns.qry.name
tshark -r capture.pcap -Y 'http.request' -T fields -e http.host -e http.request.uri
```

tcpdump — Lightweight Packet Capture

Quick and lightweight network sniffing. Save to pcap for Wireshark/tshark analysis.

PASSIVE NETWORK

```
sudo tcpdump -i en0 -w capture.pcap
sudo tcpdump -i en0 port 53           # DNS only
sudo tcpdump -i en0 host 192.168.1.1  # Specific host
```

mitmproxy — HTTPS Traffic Interception

Interactive HTTPS proxy. Intercept, inspect, modify, and replay web traffic. Essential for analyzing app communications.

ACTIVE TRAFFIC BREW INSTALL MITMPROXY

```
mitmproxy                  # Interactive TUI on port 8080
mitmweb                    # Web UI at http://localhost:8081
mitmdump -w traffic.flow   # Dump all traffic to file
mitmdump -r traffic.flow --set hardump=output.har  # Export to HAR
```

curl — HTTP Request Tool

Transfer data to/from servers. Fingerprint web servers, test APIs, follow redirects, send cookies.

ACTIVE WEB

```
curl -I https://example.com                      # Headers only (server fingerprint)
curl -L https://bit.ly/shortlink                  # Follow redirects
curl -x socks5://127.0.0.1:9050 https://example  # Via Tor
curl -w '%{http_code}' -o /dev/null -s https://example.com  # Status code only
```

DNS & Domain Intelligence (3 Tools)

whois — Domain/IP Registration Data

Query domain and IP registration data: owner, registrar, creation/expiry dates, nameservers, abuse contacts.

PASSIVE DNS

```
whois example.com | grep -i 'registrant\|created\|expires'
whois 93.184.216.34  # IP ownership
whois example.com > whois_evidence.txt  # Save for evidence
```

dig — DNS Lookup Utility

Query any DNS record type. Authoritative responses, reverse lookups, zone transfer attempts.

PASSIVE DNS

```
dig example.com MX          # Mail servers
dig example.com TXT          # SPF/DKIM/verification
dig -x 93.184.216.34         # Reverse lookup
dig axfr @ns1.example.com example.com  # Zone transfer attempt
```

nslookup — Quick DNS Query

Simple DNS query tool for quick lookups. Good for interactive use.

PASSIVE DNS

```
nslookup -type=mx example.com   # Mail exchange records
nslookup -type=txt example.com  # TXT records
nslookup -debug example.com     # Debug mode
```

Metadata & File Analysis (5 Tools)

exiftool — Metadata Extraction & Removal

Read, write, and strip metadata from images, documents, videos, and 100+ file formats. GPS coordinates in photos are one of the most valuable evidence sources in investigations.

PASSIVE METADATA BREW INSTALL EXIFTOOL

```
exiftool photo.jpg                                  # All metadata
exiftool -gps* photo.jpg                            # GPS coordinates only
exiftool -DateTimeOriginal -GPSLatitude -GPSLongitude photo.jpg
exiftool -csv -r /path/to/photos/ > metadata.csv     # Bulk CSV export
exiftool -all= photo.jpg                            # STRIP all metadata
exiftool -json photo.jpg > photo_meta.json
```

strings — Extract Hidden Text from Binaries

Extract all printable character sequences from binary files. Find embedded URLs, credentials, IPs, and emails in executables, images, and documents.

PASSIVE METADATA

```
strings -n 10 firmware.bin                          # Strings ≥10 chars
strings malware.bin | grep -i 'http\|password\|key'  # Find secrets
strings binary.bin | grep -Eo '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}'  # IPs
```

file — File Type Identification

Identify the true file type using magic bytes — never trust file extensions. Essential first step in any file forensics workflow.

PASSIVE METADATA

```
file suspicious_doc.pdf    # Is this actually a PDF?
file *                     # Identify all files in directory
file -i document.docx      # MIME type
file suspicious.jpg | grep -v JPEG  # Alert if not really a JPEG
```

xxd — Hex Dump Utility

Inspect raw bytes of any file. Essential for forensic byte-level analysis and binary patching.

PASSIVE METADATA

```
xxd -l 256 file.dat                        # First 256 bytes
xxd file.bin | grep -E 'ffd8ffe|89504e47'  # JPEG/PNG magic bytes
xxd -r modified.hex > output.bin           # Reverse: hex to binary
```

hexdump — Hex Display Tool

Display file contents in hex, decimal, octal, or ASCII with customizable formatting.

PASSIVE METADATA

```
hexdump -C file.bin | head -30              # Canonical hex+ASCII
hexdump -C -s 512 -n 64 disk.img           # Boot sector inspection
hexdump -C suspicious.bin | tail -30       # Check for appended data
```

Steganography (1 Tool)

steghide — Steganography Detection & Extraction

Embed and extract hidden data in JPEG, BMP, WAV, and AU files. Detect hidden messages in images sent between individuals.

PASSIVE STEGANOGRAPHY BREW INSTALL STEGHIDE

```
steghide info photo.jpg                     # Check for embedded data
steghide extract -sf secret.jpg -p ''      # Extract (empty password)
steghide extract -sf audio.wav -xf out.txt
```

Password Recovery (1 Tool)

john — John the Ripper (Password Cracker)

Offline password cracker supporting 200+ hash types. Used in evidence recovery when encrypted files or hashed passwords are part of a case.

PASSIVE CRYPTO BREW INSTALL JOHN

```
john --wordlist=rockyou.txt hashes.txt    # Dictionary attack
john --show hashes.txt                    # Show cracked passwords
zip2john protected.zip > zip_hash.txt && john zip_hash.txt
pdf2john doc.pdf > pdf_hash.txt && john --wordlist=rockyou.txt pdf_hash.txt
```

Digital Forensics & Recovery (2 Tools)

testdisk / photorec — Disk Recovery

Recover lost partitions, undelete files, and carve data from disk images. PhotoRec (bundled) recovers 480+ file types by signature.

ACTIVE FORENSICS BREW INSTALL TESTDISK

```
testdisk disk_image.dd              # Analyze disk image
photorec disk.img                   # Recover deleted files
testdisk /list                      # Detect drives
```

4n6notebooks — iOS Forensic Jupyter Notebooks

Bundled Jupyter notebook suite for iOS forensics: SQLCipher database decryption, Signal message parsing, ProtonMail recovery, WhatsApp/Telegram parsing, unified timeline building.

PASSIVE IOS FORENSICS

```
4n6notebooks launch            # Full JupyterLab environment
4n6notebooks launch signal     # iOS Signal parsing
4n6notebooks launch whatsapp   # WhatsApp database parser
4n6notebooks launch timeline   # Unified timeline builder
4n6notebooks export signal --format pdf  # Court-ready PDF export
```

Relationship Investigation (3 Tools)

cupidcr4wl — Adult Content & Dating Platform Crawler

Dedicated crawler for adult content and dating platforms. Accepts username or phone number. Critical for relationship investigations.

PASSIVE RELATIONSHIP

```
cupidcr4wl -u johndoe -o results.json --verbose
cupidcr4wl -p +15551234567 -o phone_report.json
cupidcr4wl -u johndoe --category hookup --proxy http://127.0.0.1:8080
```

toutatis — Instagram OSINT

Extracts hidden emails and phone numbers from Instagram accounts using an authenticated session. Often reveals contact details the account holder considers private.

PASSIVE RELATIONSHIP PIP INSTALL TOUTATIS

```
toutatis -u johndoe -s SESSION_ID -j > ig_data.json
toutatis -u johndoe -s SESSION_ID | grep -i phone
for user in $(cat ig_users.txt); do toutatis -u $user -s SESSION_ID -j; done
```

compreface — Face Recognition (Docker)

Self-hosted face recognition via Docker REST API. Detect, recognize, and verify faces across discovered photos. Requires Docker Desktop.

PASSIVE FACE RECOGNITION

```
compreface start                              # Start Docker containers
compreface detect photo.jpg                   # Detect faces
compreface recognize suspect.jpg              # Match against known subjects
compreface verify face1.jpg face2.jpg         # Compare two faces
compreface add-subject --name 'John Doe' --image ref.jpg
```

System Inspection (4 Tools)

lsof — List Open Files & Sockets

See what files, sockets, and network connections every process has open. Identify suspicious processes and their network activity.

PASSIVE SYSINFO

```
lsof -i -P -n                    # All network connections
lsof -i :8080                    # What's using port 8080?
lsof -i TCP:443 -s TCP:ESTABLISHED
lsof -i @8.8.8.8                 # Connections to specific IP
```

netstat — Network Connections & Routing

Display network connections, routing tables, and interface statistics. Quick overview of all listening ports and active connections.

PASSIVE SYSINFO

```
netstat -an | grep LISTEN         # All listening ports
netstat -an | grep ESTABLISHED    # Active connections
netstat -rn | grep default        # Default gateway
```

dtrace — Dynamic System Tracing

Trace system calls, I/O, and kernel events in real-time. Monitor processes, detect hidden activity, profile CPU usage.

PASSIVE SYSINFO

```
sudo dtrace -n 'syscall:::entry { @[execname] = count(); }'
sudo dtrace -n 'proc:::exec-success { trace(curpsinfo->pr_psargs); }'
sudo dtruss -p 12345              # Trace syscalls of a process
```

lldb — LLVM Debugger

Debug programs and inspect memory. Attach to running processes, set breakpoints, read registers and memory.

PASSIVE REVERSE ENGINEERING

```
lldb ./program              # Debug a program
lldb -p 12345               # Attach to running process
(lldb) bt                   # Full backtrace
(lldb) image list           # Loaded libraries and base addresses
```

Networking Utilities (6 Tools)

nc (Netcat), ncat (SSL netcat), traceroute, ping, wget, objdump — all pre-installed on macOS or available via Homebrew. Run `recon learn <tool>` for 10+ examples on each.

iOS Forensics (2 Tools)

keychain-decrypt — iOS Keychain Decrypter

Decrypt saved passwords, WiFi credentials, app authentication tokens, and certificates from jailbroken iOS devices. Requires jailbroken device + Xcode.

ACTIVE IOS FORENSICS

```
keychain-decrypt build-agent      # Compile on-device agent
keychain-decrypt upload-agent     # Deploy to jailbroken device
keychain-decrypt decrypt          # Full decryption workflow
keychain-decrypt filter --type wifi       # WiFi passwords only
keychain-decrypt export --format csv > keychain_items.csv
```

CHAPTER 11

SESSION TRACKING

Track multi-run investigations, resume where you left off, and maintain finding history over time.

What Sessions Do

A session is a persistent investigation record stored as a JSON file in `~/.recon/sessions/<name>.json`. Sessions allow you to:

- Resume an investigation across multiple work sessions
- Aggregate findings from multiple targets under one case
- Compare findings over time (run on Monday vs. run on Friday)
- Tag findings with case ID, examiner name, and notes
- Include the session in evidence bag outputs

Creating and Using Sessions

```
# Start a new session
recon johndoe --session investigation_001 --examiner "Agent Smith" --case-id CASE-2024-001

# Add notes to a session
recon johndoe --session investigation_001 --notes "Second scan after target changed username"

# Run a different target in the same session
recon johndoe@gmail.com --session investigation_001

# Resume a session (re-runs same targets, appends new findings)
recon --session investigation_001

# Compare current scan against a previous JSON export
recon johndoe --diff previous_scan.json
```

Session JSON Structure

~/.recon/sessions/investigation_001.json

```
{
  "session_id": "investigation_001",
  "case_id": "CASE-2024-001",
  "examiner": "Agent Smith",
  "created": "2024-01-15T09:00:00+00:00",
  "last_updated": "2024-01-15T14:30:00+00:00",
  "targets": ["johndoe", "johndoe@gmail.com"],
  "notes": ["Initial recon", "Second scan after username change"],
  "runs": [
    {
      "timestamp": "2024-01-15T09:00:00+00:00",
      "target": "johndoe",
      "tools_run": ["wmn", "sherlock", "maigret"],
      "findings_count": 47
    }
  ],
  "all_findings": [...],
  "unique_tools_used": ["wmn", "sherlock", "maigret", "holehe"]
}
```

CHAPTER 12

CHAIN OF CUSTODY & EVIDENCE BAGS

Forensically sound evidence collection that can withstand legal scrutiny.

How It Works

When you add `--evidence-bag ./evidence/ --chain-of-custody` to any scan, RECON creates a forensically structured directory containing:

📁 Evidence Bag Directory Structure

```
./evidence/
├── MANIFEST.json            ← SHA-256 hashes of ALL artifacts
├── CHAIN_OF_CUSTODY.json    ← Timestamped action log
├── reports/
│   ├── scan_report.json     ← Full unified scan report
│   └── scan_report.md       ← Markdown version
├── raw_output/
│   ├── wmn_stdout.txt       ← Raw tool output
│   ├── sherlock_stdout.txt
│   ├── holehe_stdout.txt
│   └── ...one file per tool
└── CHAIN_OF_CUSTODY.txt     ← Human-readable summary
```

Running with Evidence Bag

```
# Basic evidence bag
recon johndoe --evidence-bag ./case001/ --chain-of-custody

# Full forensic mode
recon johndoe \
  --evidence-bag ./case001/ \
  --chain-of-custody \
  --hash-verify \
  --save-raw ./case001/raw/ \
  --examiner "Det. Jane Smith" \
  --case-id CASE-2024-042 \
  -o json

# SPOTLIGHT with evidence bag
spotlight johndoe \
  --evidence-bag ./spotlight_evidence/ \
  --session case001 \
  --examiner "Investigator Jones"
```

MANIFEST.json

The manifest records every file in the evidence bag with its SHA-256 hash, size, timestamp, and originating tool:

MANIFEST.json

```
{
  "case_id": "CASE-2024-042",
  "examiner": "Det. Jane Smith",
  "created": "2024-01-15T09:00:00+00:00",
  "finalized": "2024-01-15T09:47:23+00:00",
  "artifacts": [
    {
      "filename": "scan_report.json",
      "path": "reports/scan_report.json",
      "type": "reports",
      "tool": "recon",
      "sha256": "a3f2d4e1c9b8...",
      "size_bytes": 48291,
      "timestamp": "2024-01-15T09:47:22+00:00"
    },
    ...
  ]
}
```

CHAIN_OF_CUSTODY.json

Records every action taken during the investigation with UTC timestamps:

```
CHAIN_OF_CUSTODY.json (excerpt)

{
  "case_id": "CASE-2024-042",
  "examiner": "Det. Jane Smith",
  "manifest_sha256": "c7a9b3f1...",
  "entries": [
    {"timestamp": "2024-01-15T09:00:00Z", "action": "evidence_bag_created", ...},
    {"timestamp": "2024-01-15T09:01:15Z", "action": "artifact_stored", "details": {"filename":
"wmn_stdout.txt", "sha256": "..."}},
    {"timestamp": "2024-01-15T09:47:22Z", "action": "integrity_check", "details": {"total_files":
18, "failures": 0}},
    {"timestamp": "2024-01-15T09:47:23Z", "action": "evidence_bag_finalized", ...}
  ]
}
```

Verifying Integrity

```
# Verify all hashes match — run this before presenting evidence
recon johndoe --hash-verify --evidence-bag ./case001/

# Manual verification with sha256sum
sha256sum -c <(python3 -c "
import json
m = json.load(open('./case001/MANIFEST.json'))
for a in m['artifacts']:
    print(f\"{a['sha256']}  ./case001/{a['path']}\")
")
```

✅ **Court Admissibility**

The evidence bag system is designed to meet digital forensics best practices: every artifact is SHA-256 hashed at creation, the chain of custody records every action with UTC timestamps, and the manifest is itself hashed and recorded. This creates a verifiable, tamper-evident record suitable for presentation as digital evidence in legal proceedings.

CHAPTER 13

OPSEC & ANONYMITY

Protecting your investigation identity and avoiding detection.

⚠️ **Understand Active vs. Passive**

Passive tools (flagged `passive=True` in the catalog) check public records or use indirect methods — they do not directly contact the target. **Active tools** make direct requests that may be logged by the target's server. Always use `--passive` mode when you need to avoid leaving traces.

VPN Check

```
# Refuse to run if no VPN is detected
recon johndoe --vpn-check

# RECON detects VPN by checking for these network interfaces:
# utun0, utun1, utun2 (macOS built-in VPN)
# tun0 (OpenVPN)
# wg0, wg1 (WireGuard)
```

Tor Routing

```
# Start Tor first
brew install tor
tor &

# Route all tool requests through Tor
recon johndoe --tor

# Tor + slow-down for realism
recon johndoe --tor --jitter --parallel 3 --retries 2
```

Proxy Routing

```
# HTTP proxy (Burp Suite, mitmproxy)
recon johndoe --proxy http://127.0.0.1:8080

# SOCKS5 proxy
recon johndoe --proxy socks5://127.0.0.1:1080

# SPOTLIGHT with proxy
spotlight johndoe --proxy socks5://127.0.0.1:9050
```

OpSec Check — Pre-Scan Warning

```
# Warn before running any active (trace-leaving) tools
recon johndoe --opsec-check
```

RECON will display a list of active tools in the scan and prompt you to confirm before proceeding. Passive tools are identified in the catalog by `passive=True` and are safe to run without disclosure risk.

Rate Limiting & Jitter

```
# Max 5 requests per second across all tools
recon johndoe --rate-limit 5

# Random delay between tool launches (0.5-3 seconds)
recon johndoe --jitter

# Combined — hardest to detect pattern
recon johndoe --passive --jitter --rate-limit 2 --parallel 3
```

Passive-Only Mode

```
# Run only tools that don't directly contact the target
recon johndoe --passive

# Or use the passive-only template
recon johndoe --template passive-only
```

TECHNIQUE	FLAG	PROTECTION LEVEL
Passive tools only	`--passive`	High — no direct target contact
Tor routing	`--tor`	High — hides investigator IP
VPN required	`--vpn-check`	Medium — masks real IP
Jitter delays	`--jitter`	Medium — disguises automated patterns
Rate limiting	`--rate-limit N`	Medium — avoids triggering rate limits
Parallel limit	`--parallel N`	Low-Medium — reduces burst traffic

CHAPTER 14

REPORT EXPORT

All output formats available in Project RECON.

FORMAT	FLAG	DESCRIPTION
JSON	`-o json`	Machine-readable structured output. Full findings, tool details, timestamps.
CSV	`-o csv`	Spreadsheet-compatible. Columns: site, url, status, category, tool, response_time.
HTML	`-o html`	Self-contained dark-themed report with clickable links and category grouping. Best for sharing.
PDF	`--pdf`	PDF version of the HTML report via weasyprint/wkhtmltopdf.
Markdown	`--markdown`	Markdown export for inclusion in reports or wikis.

```
# HTML report opened in browser
recon johndoe -o html --open

# JSON to specific path
recon johndoe -o json --output-file /path/to/case/report.json

# Multiple formats in one run
recon johndoe -o json --markdown --pdf

# Save raw tool output alongside report
recon johndoe -o json --save-raw ./raw_output/

# For scripting — no banner, no color, JSON output
recon johndoe -o json --no-banner --no-color | jq '.findings[].url'
```

CHAPTER 15

AUTO-UPDATE SYSTEM

Keeping all tools and site databases current.

Manual Updates

```
recon update              # Update everything
recon update --pip        # Only Python pip packages
recon update --brew       # Only Homebrew packages
recon update --bundled    # Only git-cloned bundled tools
```

Automatic Updates

The installer sets up a **launchd job** that runs every Monday at 3:00 AM and silently updates all tools. Logs are written to `~/.recon/logs/update.log`.

```
# Check auto-update status
launchctl list | grep recon

# Disable auto-updates
launchctl unload ~/Library/LaunchAgents/dev.recon.autoupdate.plist

# Re-enable auto-updates
launchctl load ~/Library/LaunchAgents/dev.recon.autoupdate.plist

# View last update log
cat ~/.recon/logs/update.log | tail -50
```

What Gets Updated

COMPONENT	UPDATE METHOD	FREQUENCY (AUTO)
WMN site database	Fetched from GitHub	Weekly
Sherlock/Maigret site data	`wmn --update`	Weekly
Python tools (pip)	`pip install --upgrade`	Weekly
Homebrew tools	`brew upgrade`	Weekly
Bundled tools (git)	`git pull` in each tool dir	Weekly

CHAPTER 16

PLUGIN SYSTEM & MARKETPLACE

Extend RECON with custom tools or browse the community marketplace.

Writing a Plugin

Create a `.py` file in `~/.recon/plugins/`. RECON loads all plugins automatically on next startup:

```
# ~/.recon/plugins/my_scanner.py
RECON_TOOLS = [
    {
        "name": "my-scanner",
        "command": "my-scanner",
        "description": "My custom scanning tool",
        "input_types": ["username", "email"],
        "args_template": ["{target}"],
        "passive": True,
        "install_hint": "pip install my-scanner",
    }
]
```

Plugin Scaffolding

```
recon plugins create my-scanner   # Creates boilerplate plugin file
recon plugins validate my.py      # Validates format and required fields
recon plugins submit my.py        # Submit to marketplace via GitHub Issue
```

Marketplace Commands

```
recon plugins                          # List installed plugins
recon plugins search                   # Browse all available plugins
recon plugins search "phone"           # Search by keyword
recon plugins install social-deep-dive # Install free plugin
recon plugins install premium-tool --key RECON-PREM-abc123  # Install paid
recon plugins uninstall my-plugin      # Remove plugin
```

💰 **Plugin Revenue Share**

The marketplace supports free and paid plugins. Paid plugin authors receive a **70/30 revenue split** — 70% to the developer. Submit your plugin, set a price, and earn money on every install.

CHAPTER 17

TROUBLESHOOTING

Solutions to the most common problems.

Command Not Found

⚠️ **"recon: command not found"**

The `recon` command is not in your PATH. Solutions:

1. Re-run the installer: `bash ~/RECON_WorkBench/install.sh`
2. Activate the virtual environment: `source ~/RECON_WorkBench/.venv/bin/activate`
3. Run directly: `python3 -m whatsmyname johndoe`
4. Add to PATH manually: `export PATH="$HOME/.local/bin:$PATH"`

Tool Not Found / Skipped

If a tool is skipped during a scan, it is not installed. Check with:

```
recon inventory            # See all tools and install status
recon learn sherlock       # Shows install command for the tool
```

Install the missing tool with the command shown in `recon learn <tool>` or `recon inventory`.

Scan Is Very Slow

- Use `--quick` for a fast scan (top 3 tools, ~60 seconds)
- Reduce timeout: `--timeout 8`
- Skip email expansion: `--no-email-expand`
- Run Tor with fewer parallel workers: `--tor --parallel 3 --timeout 30`

Many Timeouts or Errors

- Increase timeout: `--timeout 60`
- Add retries: `--retries 3`

- Some sites block automated requests — this is expected behavior
- Add `--raw` flag to see which tools are failing and why

Tor Connection Refused

- Ensure Tor is running: `tor &` or via Tor Browser
- Default port is 9050 (not 9150 from Tor Browser)
- Install required library: `pip install aiohttp-socks`
- Test Tor: `curl -x socks5://127.0.0.1:9050 https://check.torproject.org`

No Results / Zero Findings

- Run `recon update` to fetch the latest site database
- Check the username spelling — RECON searches exact-match by default
- Try `--deep` for longer timeouts on slow-responding sites
- Run `recon --list-tools` to confirm tools are installed
- Try `--nsfw` if the subject may use adult platforms

CompreFace (Face Recognition) Not Starting

- Ensure Docker Desktop is installed and running
- Run `compreface start` and wait ~60 seconds for containers to initialize
- Check status: `compreface status`
- Docker must have at least 2 GB RAM allocated

SSL Errors

- RECON disables strict SSL verification by default for self-signed certificates
- If errors persist, try routing through a local proxy: `--proxy http://127.0.0.1:8080`

Evidence Bag Hash Verification Failure

If `--hash-verify` reports mismatches, it means files in the evidence bag were modified after collection. This is a **critical finding** — the integrity of that evidence is compromised. Never modify files inside an evidence bag directory. Always work with copies.

Getting Help

```
recon --help                    # Full help text
spotlight --help                # SPOTLIGHT help
recon learn <tool>             # Tool-specific guide
recon explain "term"            # Explain a forensics concept
recon inventory                 # Full system diagnostic
```

CHAPTER 18

FORENSICS GLOSSARY

Key terms used throughout Project RECON and digital forensics investigations.

TERM	DEFINITION
OSINT	Open Source Intelligence — intelligence collected from publicly available sources including social media, websites, public records, and published data.
Chain of Custody	A chronological documentation proving the seizure, custody, control, transfer, analysis, and disposition of evidence. Each action is timestamped and logged. Breaks in chain of custody can make evidence inadmissible.
SHA-256	Secure Hash Algorithm 256-bit — a cryptographic hash function that produces a unique 64-character fingerprint of any file. If even one byte changes, the hash changes completely. Used to verify evidence has not been tampered with.
Evidence Bag	In digital forensics, a container (directory) that stores all collected artifacts alongside their hash values and collection metadata. Project RECON's EvidenceBag creates MANIFEST.json + CHAIN_OF_CUSTODY.json automatically.
Passive Reconnaissance	Gathering information without directly interacting with the target system or person. Examples: WHOIS lookups, DNS queries, social media searches. Leaves no trace on the target's systems.
Active Reconnaissance	Gathering information by directly interacting with the target. Examples: port scanning, making HTTP requests to target's servers. May leave logs on the target's systems.
Exposure Score	A calculated 1–100 risk rating reflecting how much digital footprint a target has. Higher scores indicate more publicly discoverable information. Calculated by RECON using finding counts across categories.
Enrichment	The process of enhancing raw findings with additional context — resolving IPs, performing reverse DNS lookups, adding geolocation data. Run with `--enrich`.
Correlation	Identifying relationships between findings from multiple tools — e.g., the same URL appearing in both WMN and Sherlock results. Run with `--correlate`.
Jitter	Random time delays between automated requests. Mimics human behavior and avoids triggering rate-limit detection. Enabled with `--jitter`.
Steganography	The practice of hiding secret information within ordinary, non-secret data — most commonly hiding text or files inside image files.
EXIF	Exchangeable Image File Format — metadata embedded in digital photos including camera model, settings, date/time, and critically: GPS coordinates where the photo was taken.
GAIA ID	Google Account ID — a unique numeric identifier for a Google account. Can be used to cross-reference Google Maps reviews, YouTube channel ownership, and other Google services.
Tor	The Onion Router — a network that anonymizes internet traffic by routing it through multiple encrypted relay nodes. Project RECON routes all tool requests through Tor when `--tor` is used.

TERM	DEFINITION
SOCKS5	A network proxy protocol that routes TCP connections through a proxy server. Used by Tor (`socks5://127.0.0.1:9050`) and supported by all RECON tools via `--proxy`.
Data Breach	An unauthorized exposure of private information — passwords, emails, personal data — from a hacked service. Tools like h8mail and holehe check whether a target's email appears in known breaches.
WHOIS	A query protocol that returns registration information for a domain or IP address: registrant name, organization, registrar, creation date, expiry date, and nameservers.
MX Record	Mail Exchange DNS record — specifies which mail servers handle email for a domain. Useful for identifying email providers and understanding infrastructure.
SPF / DKIM	Sender Policy Framework / DomainKeys Identified Mail — email authentication records stored as DNS TXT records. Reveal which services are authorized to send email for a domain.
Plugin	A Python file placed in `~/.recon/plugins/` that adds new tools or capabilities to Project RECON. Automatically loaded on startup.
LaunchAgent (launchd)	macOS background job scheduler. Project RECON installs a LaunchAgent (`dev.recon.autoupdate.plist`) that runs the auto-update script every Monday at 3:00 AM.

Project RECON — Rapid Evidence Collection & OSINT Network

Version 3.3.0 | License: CC BY-SA 4.0 | Platform: macOS 12+

Built on WhatsMyName by Micah Hoffman

This document contains sensitive operational information. Handle in accordance with your organization's information security policy.